Developing InterSystems Applications

Version 2026.1
2026-04-20

InterSystems Corporation One Congress Street Boston MA 02114 www.intersystems.com

Developing InterSystems Applications
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

List of Figures

List of Tables

1

Transaction Processing

transaction is a logical unit of work that groups multiple atomic operations into a single, indivisible action. An atomic operation is always fully executed in any circumstance, including if an error occurs. Typically, a transaction consists of several atomic operations executed in a specific order and treated as a single action.

This document provides an overview of transaction processing in the InterSystems IRIS® data platform. It explains how to define and manage transactions, handle errors using rollbacks, and implement strategies for disaster recovery.

1.1 About Transactions in InterSystems IRIS

In InterSystems IRIS, an *atomic operation* consists of a single operation that changes an object or row, including creation, deletion, and modification.

However, applications often require combining multiple atomic operations to complete a task. For example, consider a bank transferring money from one account to another. This task involves at least two separate operations: subtracting the transfer amount from the sender's account balance and adding the same amount to the recipient's account balance. Each of these updates is an atomic operation on its own. However, to maintain accurate financial records, they must be treated as a single unit—either both happen together, or neither happens at all. By grouping these operations within a single transaction, the system ensures that if something goes wrong after the first update but before the second, it can undo the changes, returning both accounts to their original state.

Transaction processing commands allow you to specify the sequence of operations that constitute a complete transaction. One command marks the beginning of the transaction, and after executing a series of operations, another command marks the end. If any part of the transaction fails, a rollback — either from developer-defined rollback logic or triggered automatically in cases like system failure or process termination— reverses the entire sequence.

1.2 Managing Transactions Within Applications

- Transaction Processing Commands
- Transaction Processing Details
- Examples of Transaction Processing Within Applications

1.2.1 Transaction Processing Commands

The available transaction processing commands are summarized in the following sections. These Python, SQL, and ObjectScript commands are identical in functionality, with any exceptions noted in the command definition below.

1.2.1.1 Python Transaction Commands

iris.tstart()

Begins a new transaction. Each call increases the transaction level.

iris.gettlevel()

Detects whether a transaction is currently in progress. The returned value reflects the current transaction level—the number of nested transactions opened.

Returns the current transaction level:

- >0: In a transaction; value indicates nesting level (for example, 2 means two nested **iris.tstart()** commands are active).
- 0: No transaction active

iris.tcommit()

Commits the current transaction level.

iris.trollbackone()

Rolls back changes made during the most recent nested transaction only. Outer transactions are unaffected.

iris.trollback()

Rolls back all active transactions. Resets the transaction level to 0.

Important: Using the **iris.trollback()** command can be potentially destructive, as it rolls back all active transactions for the current process. To avoid unintentionally affecting transactions beyond the one you are currently working on, InterSystems strongly recommends using **iris.trollbackone()**.

1.2.1.2 SQL Transaction Commands

InterSystems IRIS supports the ANSI SQL operations **COMMIT WORK** and **ROLLBACK WORK** (in InterSystems SQL, the keyword WORK is optional). It also supports the InterSystems SQL extensions **SET TRANSACTION**, **START TRANSACTION**, **SAVEPOINT**, and **%INTRANSACTION**.

SET TRANSACTION

Sets transaction parameters without starting a transaction.

START TRANSACTION

Begins a new transaction.

%INTRANSACTION

Determines whether a transaction is currently in progress. This command sets the *SQLCODE* variable based on the transaction state, but does not return a value.

After invoking **%INTRANSACTION**, the value of *SQLCODE* is:

- 0: A transaction is in progress.
- 100: No transaction is in progress.
- <0: In transaction, journaling disabled.

SAVEPOINT name

Marks a named point within a transaction for partial rollback.

ROLLBACK TO SAVEPOINT name

Rolls back to the named savepoint without ending the entire transaction.

ROLLBACK

Rolls back *all* changes made since the transaction began. Releases all locks and resets **%INTRANSACTION** to 0.

Important: Using the **ROLLBACK** command can be potentially destructive, as it rolls back all active transactions for the current process. To avoid unintentionally affecting transactions beyond the one you are currently working on, InterSystems strongly recommends using **ROLLBACK TO SAVEPOINT**.

COMMIT

Commits all changes made during the transaction, including those after any savepoints. **COMMIT** always finalizes the entire transaction, including any savepoints or nested transactions.

Tip: SQL does not support committing part of a nested transaction. If you use **SAVEPOINT**, do not use **COMMIT** unless you intend to finalize the entire transaction.

1.2.1.3 ObjectScript Transaction Commands

tstart

Begins a new transaction. Each call increases the transaction level.

$TLEVEL

Detects whether a transaction is currently in progress. The returned value reflects the current transaction level—the number of nested transactions opened.

Returns the current transaction nesting level.

- >0: In a transaction; value indicates nesting level (for example, 2 means two nested **tstart** commands are active).
- 0: No transaction.

tcommit

Commits the current transaction level only.

trollback 1

Rolls back the current nested transaction level only. Outer transactions are unaffected.

trollback

Rolls back all active transactions. Resets **$TLEVEL** to 0.

Important: Using the **trollback** command without an argument can be potentially destructive, as it rolls back all active transactions for the current process. To avoid unintentionally affecting transactions beyond the one you are currently working on, InterSystems strongly recommends using **trollback 1**.

1.2.1.4 Nested Transactions

While transactions should ideally be managed within a single language, it is possible to call transaction commands across languages. For example, from an SQL statement, you can call a stored procedure written in Python that uses Python transaction commands. Similarly, a Python method can call a stored procedure written in SQL that uses SQL transaction commands.

It's important to note that nested transactions behave differently depending on the language. In ObjectScript, you nest transactions by issuing **tstart** multiple times, and you can commit or roll back at specific levels using **tcommit** or **trollback 1**. In contrast, SQL uses the **SAVEPOINT** command to create nested rollback points. You can roll back to a specific savepoint using **ROLLBACK TO SAVEPOINT**, but any **COMMIT** statement ends all active transactions, including those started with **SAVEPOINT**. When mixing languages, be mindful of these behavioral differences to avoid unintended commits or rollbacks.

The following is an example of executing a SQL transaction using Python. It performs a funds transfer between two accounts, and includes a nested transaction for audit logging. If the main transfer succeeds but the audit log fails, only the logging operation is rolled back.

Python

```
def nested_transaction_example(from_id, to_id, amount):
   try:
       print("Starting outer transaction")
       iris.tstart()

       # Transfer funds
       iris.sql.exec("UPDATE Bank.Account SET Balance = Balance - ? WHERE ID = ?", amount, from_id)
       iris.sql.exec("UPDATE Bank.Account SET Balance = Balance + ? WHERE ID = ?", amount, to_id)
       print(f"Transferred {amount} from {from_id} to {to_id}")

       try:
           print("Starting nested transaction for audit logging")
           iris.tstart()

           # Log transfer (not a critical operation)
           iris.sql.exec("""
               INSERT INTO Bank.Txn (Initiator, FromAccount, ToAccount, Amount, Timestamp)
               VALUES (1, ?, ?, ?, CURRENT_TIMESTAMP)
           """, from_id, to_id, amount)

           # Simulate a failure in audit logging
           raise Exception("Simulated audit log failure")

           iris.tcommit()
           print("Audit log committed")

       except Exception as e:
           print(f"Audit logging failed: {e}")
           iris.trollbackone()
           print("Rolled back audit log only")

       iris.tcommit()
       return "Transfer completed (audit log may have failed)"

   except Exception as e:
       iris.trollbackone()
       return f"Transfer failed: {e}"
```

1.2.2 Transactions and Journaling

Transaction commands, such as begin, commit, and rollback, are recorded in the journal and can be accessed via the *Management Portal* (System Operation > Journals). Journaling plays a critical role in supporting backups and ensuring disaster recovery. Read about backups and journaling to learn more.

1.2.3 Examples of Transaction Processing Within Applications

The following are examples of transaction processing. The code below performs database modifications and then transfers funds from one account to another:

Python

```
# Transfer funds from one account to another in Python with SQL

def transfer(from_account, to_account, amount):
  try:
    iris.tstart()
    iris.sql.exec("UPDATE Bank.Account SET Balance = Balance - ? WHERE AccountNum = ?", amount,
from_account)
    iris.sql.exec("UPDATE Bank.Account SET Balance = Balance + ? WHERE AccountNum = ?", amount,
to_account)
    iris.tcommit()
    return "Transfer succeeded"
  except Exception as e:
    iris.trollbackone()
    return f"Transaction failed: {e}"
```

SQL

```
START TRANSACTION;

UPDATE Bank.Account
SET Balance = Balance - 500
WHERE AccountNum = '12345';

UPDATE Bank.Account
SET Balance = Balance + 500
WHERE AccountNum = '67890';

COMMIT;
```

Class Member

```
 ClassMethod TransferFunds(from As %String, to As %String, amount As %Double) As %Status
 {
    // Transfer funds from one account to another in ObjectScript
    tstart
    try {
        set acctFrom = ##class(Bank.Account).%OpenId(from)
        set acctTo = ##class(Bank.Account).%OpenId(to)
        if acctFrom = "" || acctTo = "" {
            throw ##class(%Exception.StatusException).CreateFromStatus($$$ERROR("Account not found"))
        }

        set acctFrom.Balance = acctFrom.Balance - amount
        set acctTo.Balance = acctTo.Balance + amount

        set status = acctFrom.%Save()
        $$$ThrowOnError(status)
        set status = acctTo.%Save()
        $$$ThrowOnError(status)

        tcommit
        return $$$OK
    } catch ex {
        trollback 1
        return ex.AsStatus()
    }
 }
```

1.3 Handling Transaction Errors with Rollbacks

1.3.1 Understanding Rollbacks

A transaction typically consists of a sequence of atomic operations that either completes entirely or not at all. If an error or system malfunction interrupts the transaction, the system uses *rollback* logic you've defined to undo any completed operations to restore the state before the failure. To cite the example of a bank transaction, rolling back an incomplete transaction prevents money from being removed from one account but not credited to another in the case of a system crash mid-process. As long as the removal of money from one account is grouped in the same transaction as depositing the money in another, a rollback ensures that each account is credited appropriately.

When developing your transaction, include a rollback command for error handling. Using structured error-handling mechanisms — such as **TRY-CATCH** in ObjectScript or **try except** in Python — is best practice. InterSystems IRIS is equipped to manage rollbacks automatically in cases of system failure or process termination. For more information, see system automated rollbacks.

1.3.2 Rollback Commands

Applications typically implement a rollback command in the error-handling block, such as CATCH in **TRY/CATCH** or EXCEPT in **TRY/EXCEPT**. To roll back the current nested level of transactions:

- *Python:* **iris.trollbackone()**
- *SQL:* **ROLLBACK TO SAVEPOINT**
- *ObjectScript:* **trollback 1**

Note: These commands will also work if called in the Terminal while a transaction runs.

1.3.2.1 Viewing Rollback Logs

After a rollback occurs, if you have enabled the *LogRollback* configuration option, the system logs the details of the rollback in the messages.log file, which you can view in the *Management Portal* (System Operation > System Logs > Messages Log).

1.3.3 Rollback Example

The following code samples illustrate how to use rollback commands within transactions, along with error handling to maintain data integrity.

Each sample starts a transaction and sets up an error handler. Operations on data structures or variables are executed, including an intentional error to trigger the rollback. If an error occurs, the handler undoes all changes and displays "Transaction Failed." If the line triggering an error were deleted and no error occurs, the transaction is committed successfully with a commit command, and a success message, “Transaction Committed," is displayed.

Python

```
def rollback_example():
    try:
        iris.tstart()
        # Withdraw too much from the account to simulate a failure
        result1 = iris.sql.exec("UPDATE Bank.Account SET Balance = Balance - 1000 WHERE AccountNum =
?", "12345")
        result2 = iris.sql.exec("UPDATE Bank.Account SET Balance = Balance + 1000 WHERE AccountNum =
?", "67890")

        # Simulate a failure if the balance drops below zero
        balance = iris.sql.exec("SELECT Balance FROM Bank.Account WHERE AccountNum = ?",
"12345").first()[0]
        if balance < 0:
            raise Exception("Insufficient funds")

        iris.tcommit()
        print("Transaction Committed")
    except Exception as e:
        iris.trollbackone()
        print(f"Transaction Failed: {e}")
```

ObjectScript

```
 TRY {
    NEW balance
    tstart
    &sql(UPDATE Bank.Account SET Balance = Balance - 1000 WHERE AccountNum = '12345')
    &sql(UPDATE Bank.Account SET Balance = Balance + 1000 WHERE AccountNum = '67890')

    &sql(SELECT Balance INTO :balance FROM Bank.Account WHERE AccountNum = '12345')
    IF balance < 0 {
        THROW ##class(%Exception.StatusException).CreateFromStatus($$$ERROR("Insufficient funds"))
    }

    tcommit
    WRITE !, "Transaction Committed"
 } CATCH ex {
    trollback 1
    WRITE !, "Transaction Failed: ", ex.DisplayString()
 }
```

Class Member

```
 ClassMethod RollbackExample() As %Status
 {
    try {
        // Open account objects
        set acctFrom = ##class(Bank.Account).%OpenId("12345")
        set acctTo = ##class(Bank.Account).%OpenId("67890")
        if (acctFrom = "" || acctTo = "") {
            throw ##class(%Exception.StatusException).CreateFromStatus($$$ERROR("Account not found"))
        }

        // Attempt fund transfer with rollback on failure
        if (acctFrom.Balance < 1000) {
            throw ##class(%Exception.StatusException).CreateFromStatus($$$ERROR("Insufficient funds"))

        }

        tstart
        set acctFrom.Balance = acctFrom.Balance - 1000
        set acctTo.Balance = acctTo.Balance + 1000

        set status = acctFrom.%Save()
        $$$ThrowOnError(status)
        set status = acctTo.%Save()
        $$$ThrowOnError(status)

        tcommit
        write "Transaction Committed",!
        return $$$OK
    } catch ex {
        trollback 1
        write "Transaction Failed: ", ex.DisplayString(),!
        return ex.AsStatus()
    }
 }
```

1.4 *Transaction Resiliency and Recovery Functionality*

- Automatic Rollbacks
- Backups and Journaling for Transaction Integrity
- Managing Concurrency with Rollbacks

1.4.1 Automatic Rollbacks

InterSystems IRIS automatically performs a rollback in cases of system failure or specific events such as process termination. Transaction rollback occurs automatically during each of the three following circumstances:

- *At the time of InterSystems IRIS startup, if recovery is needed.* When you start InterSystems IRIS and it determines that recovery is required, the system rolls back any incomplete transactions.
- **Process termination.** Halting a process using a **HALT** command (for your current process) automatically or the **^RESJOB** utility (for other running processes that are NOT your current process) affects in-progress transactions differently depending on the process type. Halting a non-interactive process (or a background job) results in the system automatically rolling back the transaction. If the process is interactive, the system displays a prompt in that process's Terminal session, asking whether to commit or rollback the transaction. This applies whether the process is halted directly or through **^RESJOB**.
- *System managers roll back incomplete transactions by running the* ***^JOURNAL*** *utility.* When you select the Restore Globals From Journal option from the **^JOURNAL** utility main menu, the journal file is restored, and all incomplete transactions are rolled back.

1.4.2 Backups and Journaling for Transaction Integrity

Journaling ensures transaction integrity by recording a time-sequenced log of database changes. Each instance of InterSystems IRIS maintains a journal that logs all **SET** and **KILL** operations made during transactions—regardless of the journal setting of the affected databases—as well as all **SET** and **KILL** operations for databases whose Global Journal State is set to "Yes."

Backups can be performed during transaction processing; however, the resulting backup file may contain partial or uncommitted transactions, which could compromise transactional consistency if restored in isolation.

In the event of a disaster that requires restoring from a backup:

1. Restore the backup file
2. Apply journal files to the restored copy of the database

Applying journal files restores all journaled updates—from the time of the backup up to the point of failure—to the recovered database. Applying journals maintains transactional integrity by completes any partial transactions and rolls back those that were not committed.

For more information, see also:

- ECP Recovery Process, Guarantees, and Limitations
- Journaling
- Importance of Journals
- Backup and Restore

1.4.3 Managing Concurrency with Rollbacks

- $INCREMENT and $SEQUENCE in Transactions and Rollbacks
- Lock Behavior with Transactions

1.4.3.1 $INCREMENT and $SEQUENCE in Transactions and Rollbacks

The primary use case for **$INCREMENT** and **$SEQUENCE** is to increment a counter before inserting new records into a database. These functions provide a fast alternative to a lock command, allowing multiple processes to increment a counter concurrently without blocking each other.

Calls to **$INCREMENT** and **$SEQUENCE** are not considered to be part of a transaction and are not journaled, regardless of whether they are invoked explicitly or implicitly—such as through **%Save()**, **_Save()**, or **CREATE TABLE**. Their effects cannot be rolled back.

Because **$INCREMENT** and **$SEQUENCE** are not journaled, rolling back a transaction does not affect the values they have allocated. If a transaction that used **$INCREMENT** is rolled back, the counter is not decremented, as adjusting the counter retroactively could disrupt other transactions, so the next use of **$INCREMENT** will pick up where the previous left off, even if the transaction that it occurred in was reverted through a rollback. This means skipped values can occur, but avoiding potential inconsistencies takes priority. Similarly, any integer values returned by **$SEQUENCE** remain allocated and unavailable to future calls, even if the transaction that assigned them was rolled back.

Note: **%Save** and **_Save** use **$INCREMENT** by default. **CREATE TABLE** uses **$SEQUENCE** by default. Whether your class uses **$SEQUENCE** or **$INCREMENT** is defined in the IdFunction Storage Keyword, which can be configured as needed.

1.4.3.2 Lock Behavior with Transactions

Releasing a lock (**iris.lock()** in Python, **LOCK** in ObjectScript or **LOCK TABLE** in SQL) during a transaction may result in one of two possible states: the lock is fully released and immediately available to other processes, or it may enter a *delock* state. In a delock state, the lock behaves as released within the current transaction, allowing further lock operations on the same resource from within that transaction. However, to other processes, the lock remains active and unavailable until the transaction is either committed or rolled back—at which point the lock is fully released. To avoid locking conflicts, be mindful of when locks are released during a transaction and monitor for delock status using the *Monitor Locks* page.

Additionally, when configuring a lock, you can specify a timeout. If a lock attempt times out, the system sets the value of **$TEST**, which reflects the outcome of the lock attempt but is not affected by a later rollback of the transaction.

For more information about delock states, lock behavior, and best practices, refer to Managing Transactions and Locking with Python and Lock Management.

1.5 Advanced and Legacy Transaction Controls

- Suspending All Current Transactions
- The Legacy Utility ^%ETN and Transactions

1.5.1 Suspending All Current Transactions

You can temporarily suspend all current transactions within a process using the **TransactionsSuspended()** method. Changes made while transactions are suspended cannot be rolled back. Changes made before or after the suspension are still able

to be rolled back. This is a potent feature that should be used with caution. If used recklessly, it can lead to incomplete and irreversible changes, which may affect data integrity. Use it only when rollback behavior is not needed and data consistency is not at risk. Suspending transactions can be appropriate in specific, controlled cases—such as bypassing rollback for audit logging, improving performance on low-risk operations, or ensuring certain changes persist during administrative tasks.

Important: If a global is modified during a transaction and then modified again while transactions are suspended, rolling back the transaction may result in an error. To prevent rollback errors while suspending transactions, avoid modifying the same global both inside a transaction and again while that transaction is suspended. If such a conflict is possible, use application-level safeguards—like a lock—to coordinate access and ensure the global isn't changed during suspension. The safest approach is to isolate operations that require transaction suspension from those that rely on rollback behavior.

To suspend all current transactions, invoke one of the following methods:

- In Python, call the **TransactionsSuspended()** method of the iris.system.Process class. This method takes a boolean argument: 1 suspends all current transactions and 0 (default) resumes them. It returns a boolean indicating the previous state.
- In ObjectScript, call the **TransactionsSuspended()** method of the %SYSTEM.Process class. This method takes a boolean argument: 1 suspends all current transactions and 0 (default) resumes them. It returns a boolean indicating the previous state.

There is no SQL equivalent to **TransactionsSuspended()**.

1.5.2 The Legacy Utility ^%ETN and Transactions

While **^%ETN** remains functional for compatibility with legacy systems, you should use structured exception handling and explicit rollback commands in modern applications. Further details are provided in the ^%ETN documentation.

^%ETN is a legacy utility that remains available for handling incomplete transactions in certain systems. If an error occurs during a transaction and you have not explicitly handled rollback using a rollback command, **^%ETN** or **FORE^%ETN** will prompt the user to commit or rollback the transaction. Committing an incomplete transaction can compromise logical database integrity. To prevent this, use structured error-handling mechanisms as recommended in the section on error handling alongside explicit rollback commands.

If your application invokes **^%ETN** or **FORE^%ETN** in an interactive process (such as running a routine in the Terminal) after an error with an active transaction, the user sees the following prompt before the process terminates:

```
You have an open transaction.
Do you want to perform a (C)ommit or (R)ollback?
R =>
```

If the user do not respond within 30 seconds, the system automatically rolls back the transaction. In a background job, the rollback happens immediately without displaying a prompt.

By default, **^%ETN** and **FORE^%ETN** exit using **HALT**. In a background job, **HALT** automatically rolls back the transaction. In an interactive process, **HALT** prompts the user to either commit or rollback. However, the **BACK^%ETN** and **LOG^%ETN** entry points do display a prompt or automatically roll back failed transactions; the user must explicitly roll back using **trollback 1** before calling these routines.

2

Locking and Concurrency Control

An important feature of any multi-process system is concurrency control, which prevents multiple processes from modifying a single data element simultaneously, thereby preventing data corruption. Consequently, InterSystems IRIS® data platform provides a lock management system. This page provides an overview.

2.1 Locking Overview

Locking prevents different processes from changing the same element of data at the same time. The basic locking mechanism is a lock command, **LOCK** in ObjectScript and **iris.lock()** in Python, which delays activity in one process until another process signals that it is permitted to proceed. InterSystems SQL also includes locking behavior and controls for data access; see Locking in SQL and Objects.

In InterSystems IRIS, locking requires that mutually competing processes use the same lock names. Take this scenario:

1. Process A issues a lock command on a global, and InterSystems IRIS creates an exclusive lock on that global. Process A then makes changes to nodes in a global.
2. Process B issues a lock command on the same global with the same lock name. When process B finds out that an exclusive lock exists, it pauses. The lock call does not return, and no additional lines of code can be executed.
3. When process A releases the lock, process B's lock command finally returns, and process B continues. Process B can now modify the nodes in the global.

This example illustrates the importance of lock names. Processes must use the same names to coordinate access to shared data. For details on choosing and structuring lock names, see Naming Locks.

2.2 How Locks Work: Anatomy & Lifecycle

This section defines the components that make up a lock in InterSystems IRIS. Every lock has the following elements, established at the time of acquisition, that determine how it functions and at what scope:

Lock name

The identifier of the resource being protected. By convention, names mirror the global or node (for example, `^MyGlobal(1)`). Competing processes must use the *same* name to coordinate. See Naming Locks.

Mode (shared vs. exclusive)

Controls contention: shared allows concurrent readers; exclusive blocks all other locks of that name. Exclusive is the default. See Exclusive and Shared Locks.

Acquisition style (incremental vs. simple)

Determines how existing locks are handled when acquiring new ones. Incremental adds to what you already hold (reference-counted); simple replaces the current set. Python locks are always incremental; in ObjectScript, `LOCK +name` is incremental and `LOCK name` is simple. See Incremental and Simple Locks.

Timeout

The duration a process should wait for a lock request to succeed before timing out, with 0 indicating non-blocking. See Locks with Timeouts.

When your process requests a lock, the following occurs:

1. InterSystems IRIS enqueues it for the name.
2. If a conflicting lock exists (based on the requested mode), your process waits up to the timeout; otherwise, it acquires immediately.
3. On success, your code proceeds; on timeout, handle it per language: in ObjectScript, check **$TEST**; in Python, catch the IRISTimeoutError exception.
4. Your process releases the lock, or it is released automatically when the process ends (visible in the lock table).

2.3 Core Locking Concepts

- Naming Locks
- Locks with Timeouts
- Incremental and Simple Locks
- Managing the Lock Table

2.3.1 Naming Locks

One of the arguments for a lock command is the lock name. Lock names are arbitrary, but by universal convention, programmers use names identical to the items they lock. Usually, the item to be locked is a global or a node of a global. Lock names typically follow the same naming conventions as local and global variables, including case sensitivity and the use of subscripts. Find out more in Variables.

CAUTION: Do not use process-private global names as lock names (you would not need such a lock anyway because, by definition, only one process can access such a global).

Competing processes need to use the same lock name when accessing shared data. In InterSystems IRIS, locking is not enforced at the data level; it is enforced by agreement between processes that use the same lock name. If two processes use different lock names for the same global or data structure, they can both acquire locks independently and modify the same data without coordination. To prevent conflicts, establish clear naming conventions for locks and apply them consistently across all code that accesses shared data. Lock names should be meaningful and clearly identify the global or node being locked.

Tip: Since lock naming is a matter of convention and lock names are arbitrary, it is not necessary to define a variable before creating a lock with the same name.

2.3.1.1 Lock Names and Performance

The form of the lock name affects performance because of how InterSystems IRIS allocates and manages memory. Locking is optimized for lock names that use subscripts. An example is `^name("ABC")`.

In contrast, InterSystems IRIS is not optimized for lock names such as `^nameABC` or `^nameDEF`. Non-subscripted lock names can also cause ECP-related performance problems.

For a visual walk-through of how locking a subscripted node implicitly affects its ancestors and descendants, see Example: Locking Arrays and Subnodes.

2.3.2 Locks with Timeouts

Timeouts specify the duration a process should wait for a lock request to succeed before timing out. If a lock cannot be applied within the specified timeout period, the process will stop waiting, and the lock request will fail. Timeouts are especially relevant to incremental locks to avoid deadlocks.

A timeout does the following:

1. Attempts to add the given lock to the lock table. The lock is added to the lock queue, if one exists.
2. Pauses execution until the lock is acquired or the timeout period ends, whichever comes first.
3. *For ObjectScript* **LOCK***:* Sets the value of the **$TEST** special variable. If the lock is acquired, InterSystems IRIS sets **$TEST** equal to 1. Otherwise, InterSystems IRIS sets **$TEST** equal to 0.

   For Python **iris.lock()***:* Raises IRISTimeoutError if the timeout period elapses before the lock is acquired; otherwise, returns normally.

See Creating Locks with Timeouts for examples and more.

2.3.3 Incremental and Simple Locks

A lock can be acquired in two basic modes: incremental or simple. These modes control how a process manages existing locks when acquiring new ones.

2.3.3.1 Incremental Locks

An incremental lock adds a new entry for the specified name without releasing any existing locks. Each acquisition increments a reference count. The lock is only released when the count decrements to zero, preventing other processes from acquiring it until then.

Note: By default, all Python locks are incremental.

Python

```
# Exclusive, non-escalating lock, no wait
iris.lock("", 0, "^MyGlobal", 1)
```

ObjectScript

```
LOCK +^MyGlobal(1)
```

Incremental locks are the standard way to protect critical sections because they let you hold multiple locks at once, and acquire them at different times during execution. They are best combined with timeouts to avoid deadlocks.

2.3.3.2 Simple Locks

In ObjectScript, simple locks are atomic replacements of the held set; they're rarely used outside small critical sections or legacy code. A simple lock replaces all existing locks held by the process with the new one(s). Unlike incremental locks, it does not maintain a reference count across acquisitions.

ObjectScript

```
LOCK (^MyVar1,^MyVar2,^MyVar3)
```

Simple locks are less common because applications usually need to acquire multiple locks at different stages of processing. However, you may specify lock types and timeouts with simple locks.

Note: Simple locks are ObjectScript only. They cannot be used in Python.

2.3.4 Managing the Lock Table

InterSystems IRIS maintains a system-wide, in-memory table that records all current locks and the processes that own them. This table, the lock table, is accessible via the Management Portal (**System Operation** > **Locks** > **Manage Locks**), where you can view the locks and (in rare cases, if needed) remove them. Note that a given process can own multiple locks with different names (or even multiple locks with the same name).

When a process ends, the system automatically releases all locks it owns. Thus, it is not generally necessary to remove locks via the Management Portal, except in cases of an application errors.

Note: When viewing locks in the Management Portal, the **Directory** column shows the database path to which the lock applies. This is particularly useful for understanding locks acquired across namespaces or through global mappings.

2.3.4.1 Lock Table Limits and Tuning

The lock table cannot exceed a fixed size, which you can specify using the **locksiz** setting. For information, see Monitoring Locks. Consequently, the lock table may fill up, preventing further locks. See Lock Table Full for more information.

Note: Implicit locks are not included in the lock table and thus do not affect its size. For an example of implicit locks created by locking array nodes, see Example: Locking Arrays and Subnodes.

2.3.4.2 Lock Table Full

If the lock table reaches capacity, InterSystems IRIS writes the following message to the messages.log file:

```
LOCK TABLE FULL!!!
```

InterSystems IRIS also writes a locktablefull.log file in the instance `mgr` directory. This file contains diagnostic information captured at the time the condition occurs, including lock table memory usage and a snapshot of lock entries. You can use this file to help identify the processes or lock patterns that contributed to the lock table filling up.

Filling the lock table is not generally considered to be an application error; InterSystems IRIS also provides a lock queue, and processes wait until there is space to add their locks to the lock table.

2.4 Lock Types and Variations

Lock type codes modify a lock's behavior at the moment you acquire it. There are four lock type codes, shown below; they are not case-sensitive.

- `S` - Adds a shared lock. See Exclusive and Shared Locks.
- `E` - Adds an escalating lock. See Non-Escalating and Escalating Locks.
- `I` - Adds immediate unlock timing to the lock. See Deferred and Immediate Unlocks.
- `D` - Adds deferred unlock timing to the lock. See Deferred and Immediate Unlocks.

Note: Unlocks types are ObjectScript only. They cannot be used with Python.

This documentation gives an overview of lock type behavior. See ObjectScript reference for further detail on lock types.

These lock type codes can be combined (as is the case with `EI`, produces the effect of both `E`, escalating and `I`, immediate unlock, thus creating an exclusive escalating lock with immediate unlock).

Table 2–1: Lock Type Summary

	Exclusive Locks	Shared Locks (#"S" locks)
Non-escalating Locks	• **`locktype omitted`** - Default lock type • `"I"` - Exclusive lock with immediate unlock • `"D"` - Exclusive lock with deferred unlock	• `"S"` - Shared lock • `"SI"` - Shared lock with immediate unlock • `"SD"` - Shared lock with deferred unlock
Escalating Locks (#"E" locks)	• `"E"` - Exclusive escalating lock • `"EI"` - Exclusive escalating lock with immediate unlock • `"ED"` - Exclusive escalating lock with deferred unlock	• `"SE"` - Shared escalating lock • `"SEI"` - Shared escalating lock with immediate unlock • `"SED"` - Shared escalating lock with deferred unlock

2.4.1 Assigning Lock Types

To assign a lock type:

Python

```
iris.lock(lock_mode, timeout, ^+lockReference, subscripts)
```

ObjectScript

```
LOCK +lockname#locktype
```

Where *lock_mode* or *Locktype* is one or more lock type codes, in any order, enclosed in double quotes for addition (or removal). In ObjectScript, a pound character (#) must separate the lock name from the lock type.

2.4.2 Exclusive and Shared Locks

Any lock is either exclusive (the default) or shared (`S`). These types have the following significance:

- While one process owns an exclusive lock (with a given lock name), no other process can acquire any lock with that lock name.
- While one process owns a shared lock (with a given lock name), other processes can acquire shared locks with that name, but no process can acquire an exclusive lock with that name.

The typical purpose of an exclusive lock is to indicate that you intend to modify a value and that other processes should not attempt to read or modify that value until you are done. The typical purpose of a shared lock is to indicate that you intend to read a value and that other processes should not attempt to modify that value; they can, however, read the value.

For a detailed example, see Example: Protecting Application Data.

2.4.3 Non-Escalating and Escalating Locks

Any lock is either non-escalating (the default) or escalating (`E`). These types determine how InterSystems IRIS manages memory and lock tracking when your application holds many locks on related nodes. They have the following significance:

- For non-escalating locks, each node you lock is tracked individually in the lock table. This gives precise control, but can consume memory if you lock many nodes.
- For escalating locks, when a process locks more than a specific number (by default, 1,000) of parallel nodes at the same subscript level, InterSystems IRIS automatically "escalates" them. It replaces the individual node locks with a single lock on the parent node, implicitly locking the entire branch. Releasing child node locks decrements the count. When enough locks are removed, InterSystems IRIS automatically removes the parent-level lock.

Note: Escalation applies only to subscripted lock names. Attempting to escalate a flat name results in a `<COMMAND>` error. The escalation threshold is configurable via LockThreshold.

The typical purpose of an escalating lock is to manage large numbers of locks without overwhelming the lock table. By contrast, use non-escalating locks when fine-grained concurrency and memory usage are not a concern.

For a detailed example, see Example: Escalating Lock.

2.4.4 Deferred and Immediate Unlocks

The lock type codes `I` (immediate) and `D` (deferred) control *when* a lock is released relative to transactions. These options adjust unlock timing only; they do not change the lock name or mode. They are not case-sensitive and cannot be used together for the same lock operation.

Note: These lock types are only available in ObjectScript, not Python.

- Immediate unlock releases the lock as soon as the unlock is issued, regardless of whether a transaction is active. Use when exclusive access is no longer required, but the transaction continues for other work.
- Deferred unlock schedules the unlock to occur at the end of the current transaction (commit or rollback). Use to keep the resource protected until the transaction reaches a durability boundary.

For syntax, see Assigning Lock Types. For end-to-end demonstrations, see Example: Immediate Unlock and Example: Deferred Unlock. Related timing behavior is shown in Example: Timed Lock.

2.5 Locking Syntax and Examples

2.5.1 Adding and Removing Locks

To add a lock, use a lock command as follows:

Python

```
iris.lock(lockMode, timeout, lockReference, *subscripts)
```

ObjectScript

```
 LOCK +lockname#locktype :timeout
```

There are different types of locks, each with distinct behaviors. With *lock_mode* or *#locktype*, you can specify the lock variation. Learn more about the lock types available in Lock Types.

In the above example, you can specify a *timeout*, where *timeout* is the timeout period in seconds. If you specify a timeout, this lock becomes an Incremental Lock with a Timeout. A timeout is an effective way to reduce the risk of deadlock. If you specify *timeout* as 0, InterSystems IRIS makes a single attempt to add the lock. In Python, a lock attempt that times out raises IRISTimeoutError; in ObjectScript, **$TEST** is set to 0.

Tip: Hold locks only as long as necessary. Keeping them too long increases the risk of contention and performance bottlenecks.

2.5.1.1 Unlocking All Locks

To remove all locks held by the current process:

Python

```
 iris.releaseAllLocks()
```

ObjectScript

```
 LOCK
```

For both Python and ObjectScript, these contain no arguments.

Unlocking all locks is not common practice. It is best to release specific locks as soon as possible. Locks are automatically released when their related process ends. Avoid using this in shared processes or servers, as it can remove unrelated locks held by helpers within the same process.

2.5.1.2 Removing Typed Locks

To remove a lock of a specific type:

Python

```
iris.unlock(lock_list, timeout_value=None, locktype=None)
```

ObjectScript

```
 LOCK -lockname#locktype
```

Examples:

Python

```
# Remove one shared lock on ^G(1)
iris.unlock(['^G(1)'], None, "S")

# Remove one escalating lock on ^G(1)
iris.unlock(['^G(1)'], None, "E")

# Remove one exclusive, non-escalating lock on ^G(1)
iris.unlock(['^G(1)'])
```

ObjectScript

```
 LOCK -^G(1)#"S"   ; removes one shared lock
 LOCK -^G(1)#"E"   ; removes one exclusive escalating lock
 LOCK -^G(1)#"SD"  ; removes one shared lock with deferred unlock
```

2.5.2 Creating Incremental Locks

An incremental lock allows you to apply the same lock multiple times, effectively incrementing the lock count. By default, all locks made in Python are incremental.

To add an incremental lock:

Python

```
 # Python locks are always incremental; here with 0-second timeout (nonblocking)
iris.lock("", 0, "^Customer", 1234)
```

ObjectScript

```
 LOCK +lockname
```

A process can add multiple incremental locks with the same name; these locks can be of different types or the same type. Learn more about lock types in Lock Types.

To learn more about incremental locks, see Incremental Locks.

2.5.3 Creating Locks with Timeouts

Timeouts specify the duration a process should wait for a lock request to succeed before timing out.

The following are examples of incremental locks with timeouts:

Python

```
iris.lock(lock_mode, timeout, ^+lockReference, subscripts)
```

ObjectScript

```
 LOCK +lockname#locktype :timeout
```

Where *timeout* or *:timeout* is the timeout period in seconds. In ObjectScript, the space before the colon is optional. If you specify a timeout as 0, InterSystems IRIS will attempt to add a lock.

Note: If you try to take a lock on a parent node with a 0 timeout and already have a lock on a child node, the zero timeout is ignored, and an internal 1–second timeout is used instead.

If you're using a timeout argument, you may be advised to build in a check for the value of the **$TEST**; in Python, wrap **iris.lock()** in a "try/except" block and catch IRISTimeoutError if the lock cannot be acquired in time. The following shows an example:

Python

```
try:
    # Try up to 2 seconds to acquire an exclusive, non-escalating lock
    iris.lock("", 2, "^ROUTINE", routinename)
    # ... protected work ...
finally:
    # Release using the same name
    iris.unlock([f'^ROUTINE("{routinename}")'])
```

ObjectScript

```
 Lock +^ROUTINE(routinename):0
 If '$TEST {  Return $$$ERROR("Cannot lock the routine: ",routinename)}
```

2.6 Namespaces and Locking

Locks are typically used to control access to globals. Because a global can be accessed from multiple namespaces, InterSystems IRIS provides automatic cross-namespace locking support. The behavior is automatic and needs no intervention. There are several scenarios to consider when understanding the implications of this:

- Every namespace has one or more default databases, which contain data for persistent classes and any additional globals; this is the *globals database* for this namespace. When you access data (in any manner), InterSystems IRIS retrieves it from this database unless other considerations apply. A given database can serve as the globals database for more than one namespace. See Example: Multiple Namespaces with the Same Globals Database.
- A namespace can include mappings that provide access to globals stored in other databases. See Example: Namespace Uses a Mapped Global.
- A namespace can include subscript level global mappings that provide access to globals partly stored in other databases. See Example: Namespace Uses a Mapped Global Subscript.
- Code running in one namespace can use an extended reference to access a global that is not otherwise available in that namespace. See Example: Extended Global References.

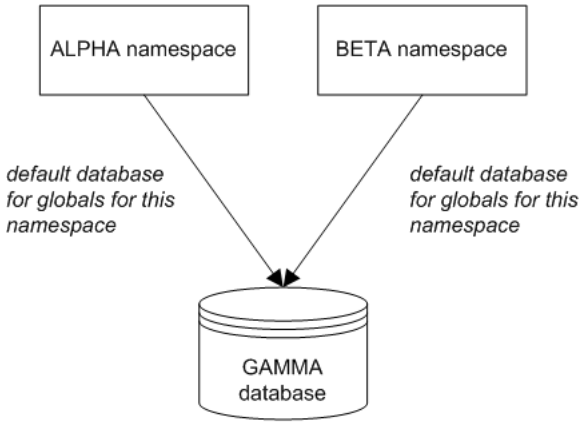
Although lock names are intrinsically arbitrary, when you use a lock name that starts with a caret (^), InterSystems IRIS provides special behavior appropriate for these scenarios. The following subsections give the details. For simplicity, only exclusive locks are discussed; the logic is similar for shared locks.

2.6.1 Example: Multiple Namespaces with the Same Globals Database

While one process holds an exclusive lock with a given lock name, no other process can acquire a lock with that name.

If the lock name starts with a caret, this rule applies to *all* namespaces that use the same globals database as the locked process.

For example, suppose the namespaces ALPHA and BETA are both configured to use database GAMMA as their globals database. The following shows a sketch:



Then consider the following scenario:

1. In namespace ALPHA, process A acquires an exclusive lock named ^MyGlobal(15).
2. In namespace BETA, process B tries to acquire a lock with the name ^MyGlobal(15). This lock command does not return; the process is blocked until process A releases the lock.

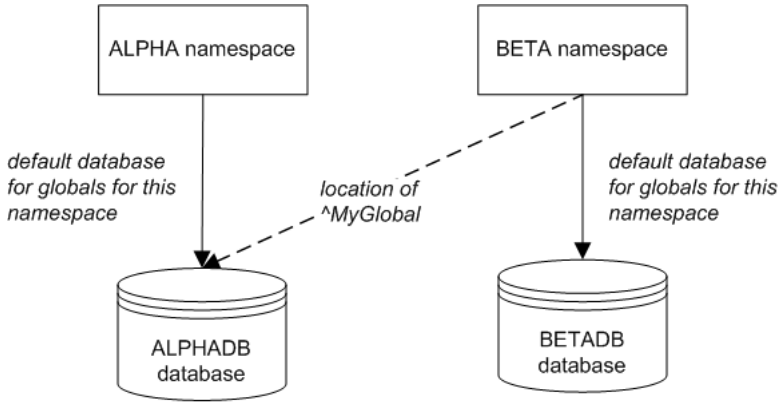
In this scenario, the lock table contains only the entry for the lock owned by process A. If you examine the lock table, you will notice that the *Directory* column indicates the *database* to which this lock applies. For example:

Owner	ModeCount	Reference	Directory
1284	Exclusive	^%SYS("CSP","Daemon")	c:\intersystems\iris\mgr\
26324	Exclusive	^ISC.LMFMON("License Monitor")	c:\intersystems\iris\mgr\
23400	Exclusive	^ISC.Monitor.System	c:\intersystems\iris\mgr\
23180	Exclusive	^TASKMGR	c:\intersystems\iris\mgr\
19776	Exclusive_e	^MyGlobal(15)	c:\intersystems\iris\mgr\gammadb\
23948	Exclusive	^%cspSession("vgMJ4iLMCL")	c:\intersystems\iris\mgr\irislocaldata\

2.6.2 Example: Namespace Uses a Mapped Global

If one or more namespaces have global mappings, InterSystems IRIS automatically enforces the lock mechanism across all applicable namespaces. The system automatically creates additional lock table entries when locks are acquired in the non-default namespace.

For example, suppose that namespace ALPHA is configured to use database ALPHADB as its globals database. Suppose that namespace BETA is configured to use a different database (BETADB) as its globals database. Namespace BETA also includes a global mapping that specifies that ^MyGlobal is stored in the ALPHADB database. The following shows a sketch:



Then consider the following scenario:

1. In namespace ALPHA, process A acquires an exclusive lock with the name ^MyGlobal(15).

   The lock table contains only the entry for the lock owned by process A. This lock applies to the ALPHADB database:

   ```
   19776   Exclusive_e   ^MyGlobal(15)                    c:\intersystems\iris\mgr\alphadb\
   ```

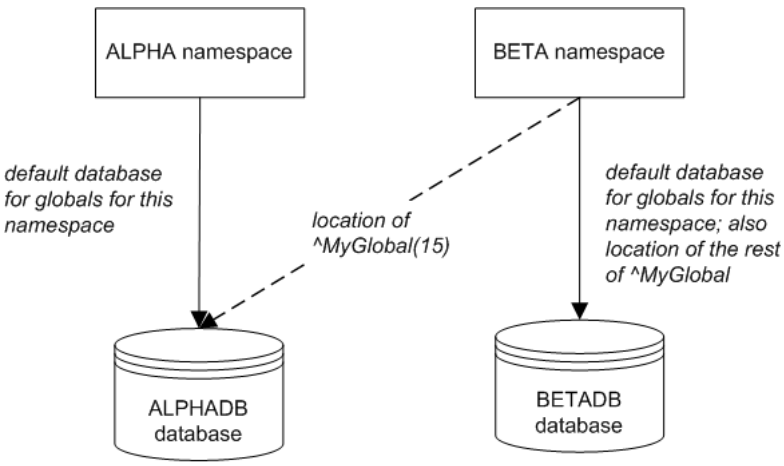
2. In namespace BETA, process B tries to acquire a lock with the name ^MyGlobal(15). The lock command does not return; the process is blocked until process A releases the lock.

2.6.2.1 Example: Namespace Uses a Mapped Global Subscript

If one or more namespaces have global mappings that use subscript level mappings, InterSystems IRIS automatically enforces the lock mechanism across all applicable namespaces. The system automatically creates additional lock table entries when locks are acquired in a non-default namespace.

For example, suppose that namespace ALPHA is configured to use the database ALPHADB as its globals database. Namespace BETA uses the BETADB database as its globals database.

Also suppose that the namespace BETA also includes a subscript-level global mapping so that ^MyGlobal(15) is stored in the ALPHADB database (while the rest of this global is stored in the namespace's default location). The following shows a sketch:



Then consider the following scenario:

1. In namespace ALPHA, process A acquires an exclusive lock with the name ^MyGlobal(15).

As with the previous scenario, the lock table contains only the entry for the lock owned by Process A. This lock applies to the ALPHADB database (for example, `c:\InterSystems\IRIS\mgr\alphadb`).

2. In namespace BETA, process B tries to acquire a lock named `^MyGlobal(15)`. This lock command does not return; the process is blocked until process A releases the lock.

When a non-default namespace acquires a lock, the overall behavior remains the same; however, InterSystems IRIS handles the details slightly differently. Suppose that in namespace BETA, a process acquires a lock with the name `^MyGlobal(15)`. In this case, the lock table contains two entries, one for the ALPHADB database and one for the BETADB database. The process in the BETA namespace owns both locks. Releasing the name in BETA removes both entries automatically.

19776	Exclusive_e	^MyGlobal(15)	c:\intersystems\iris\mgr\alphadb\
19776	Exclusive_e	^MyGlobal(15)	c:\intersystems\iris\mgr\betadb\

When this process releases the lock name `^MyGlobal(15)`, the system automatically removes both locks.

2.6.3 Example: Extended Global References

Code running in one namespace can use an extended reference to access a global that is not otherwise available in that namespace. In this case, InterSystems IRIS adds an entry to the lock table that affects the relevant database. The lock is owned by the process that created it. For example, consider the following scenario. For simplicity, there are no global mappings in this scenario.

1. Process A is running in the ALPHA namespace, and this process uses the following command to acquire a lock on a global that is available in the BETA namespace:

   Python

   ```
   # Lock ^["beta"]MyGlobal(15) from the current namespace
   iris.lock("", 0, '^["beta"]MyGlobal', 15)
   ```

   ObjectScript

   ```
   lock ^["beta"]MyGlobal(15)
   ```

2. Now the lock table includes the following entry:

19776	Exclusive_e	^MyGlobal(15)	c:\intersystems\iris\mgr\betadb\

   Note that this shows only the global name (rather than the reference used to access it). Also, in this scenario, BETADB is the default database for the BETA namespace.

3. In namespace BETA, process B tries to acquire a lock with the name `^MyGlobal(15)`. This lock command does not return; the process is blocked until process A releases the lock.

A process-private global is technically an extended reference, but InterSystems IRIS does not support using a process-private global name as a lock name; you would not need such a lock anyway because, by definition, only one process can access such a global.

2.7 Avoiding Deadlocks

Incremental locking is potentially dangerous because it can lead to a situation known as *deadlock*. This situation occurs when two processes each assert an incremental lock on a variable already locked by the other process. Because the attempted

locks are incremental, the existing locks are not released. As a result, each process hangs while waiting for the other process to release the existing lock.

As an example:

1. Process A issues this command:

   Python

   ```
   iris.lock("", 0, "^MyGlobal", 15)
   ```

   ObjectScript

   ```
   lock +^MyGlobal(15)
   ```

2. Process B issues this command:

   Python

   ```
   iris.lock("", 0, "^MyOtherGlobal", 15)
   ```

   ObjectScript

   ```
   lock +^MyOtherGlobal(15)
   ```

3. Process A issues this command:

   Python

   ```
   iris.lock("", 0, "^MyOtherGlobal", 15)
   ```

   ObjectScript

   ```
   lock +^MyOtherGlobal(15)
   ```

   This lock command does not return; the process is blocked until process B releases this lock.

4. Process B issues this command:

   Python

   ```
   iris.lock("", 0, "^MyGlobal", 15)
   ```

   ObjectScript

   ```
   lock +^MyGlobal(15)
   ```

   This lock command does not return; the process is blocked until process A releases this lock. Process A, however, is blocked and cannot release the lock. Now, these processes are waiting for each other.

Deadlock is considered an application programming error and should be prevented. There are several ways to prevent deadlocks:

- Always include the timeout argument.
- Follow a strict protocol for the order in which you issue incremental lock commands. Deadlocks cannot occur as long as all processes follow the same order for lock names. A simple protocol is to add locks in collating sequence order.
- In ObjectScript, use simple locking rather than incremental locking; that is, do not use the + operator. As noted earlier, with simple locking, the **LOCK** command first releases all previous locks held by the process. (In practice, however, simple locking is not often used.)

If a deadlock occurs, you can resolve it by using the Management Portal or the **^LOCKTAB** routine. See Monitoring Locks.

2.8 Locking in SQL and Objects

When you work with InterSystems SQL or persistent classes, you do not need to use the ObjectScript **LOCK** or Python **iris.lock()** command directly because there are alternatives suitable for your use cases. (Internally, these alternatives all use an ObjectScript **LOCK** command.)

- InterSystems SQL provides commands for working with locks. For details, see the InterSystems SQL Reference. Similarly, the system automatically performs locking on **INSERT**, **UPDATE**, and **DELETE** operations (unless you specify the %NOLOCK keyword).
- The %Persistent class provides a way to control concurrent access to objects, namely, the concurrency argument to **%OpenId()** and other methods of this class. All persistent objects inherit these methods. See Object Concurrency.

  The %Persistent class also provides the methods **%GetLock()**, **%ReleaseLock()**, **%LockId()**, **%UnlockId()**, **%LockExtent()**, and **%UnlockExtent()**. For details, see the class reference for %Persistent.

2.9 See Also

- Locking Examples for more detailed examples of locking in practice.
- LOCK ObjectScript command reference.
- iris.lock() Python command reference.
- ^$LOCK (**^$LOCK** is a structured system variable that contains information about locks.)
- Transaction Processing
- Details of Lock Requests and Deadlocks
- Managing the Lock Table
- Monitoring Locks

3

Locking Examples

InterSystems IRIS® data platform provides a lock management system for concurrency control. This page presents examples that demonstrate when and how to use locks to protect data and coordinate activities. For an overview of concepts, see Locking and Concurrency Control.

3.1 Example: Protecting Application Data

Application data in InterSystems IRIS is stored in globals, which can be accessed by many processes simultaneously. Without coordination, simultaneous reads and updates can lead to conflicts or partial changes. Locks provide a way to control access: before an application reads or modifies a piece of data, it can establish one or more locks to prevent other processes from interfering. This guarantees data consistency and predictable behavior. For example:

- When an application needs to read one or more global nodes without allowing other processes to modify their values during the read operation, it uses shared locks for those nodes.
- When an application needs to modify one or more global nodes without allowing other processes to read them during the modification, it uses exclusive locks for those nodes.

After the locks are in place, the application performs the read or modification. Once finished, the locks are released so that other processes can proceed.

3.2 Example: Preventing Simultaneous Activity

Some activities must never run in parallel - think scheduled jobs, batch exports, or maintenance tasks. If two processes start the same routine at the same time, the results can include duplicated work, inconsistent state, or wasted resources. To prevent this, routines coordinate using a lock and a small bit of application state in a global.

For example, consider a routine (`^NightlyBatch`) that must be single-instance. In this pattern, the global records “in-progress” metadata for internal coordination rather than business data.

At a very early stage, the routine:

1. Attempts to acquire an exclusive lock on a specific global node (for example, `^AppStateData("NightlyBatch")`) with a timeout.
2. If the lock is acquired, set nodes in a global to record that the routine has been started (as well as any other relevant information); otherwise, exits with a message indicating another instance is already running. For example:

Python

```
appstate = iris.gref("^AppStateData")
appstate["NightlyBatch"] = 1
appstate["NightlyBatch", "user"] = getpass.getuser()
```

ObjectScript

```
 set ^AppStateData("NightlyBatch")=1
 set ^AppStateData("NightlyBatch","user")=$USERNAME
```

Then, at the end of its processing, the same routine would clear the applicable global nodes and release the lock.

The following partial example demonstrates this technique, which is adapted from code that InterSystems IRIS uses internally:

Python

```
import time
import getpass
import iris

appstate = iris.gref("^AppStateData")
try:
    # Non-blocking attempt (0-second timeout)
    iris.lock("", 0, "^AppStateData", "NightlyBatch")
except Except as e:
    # Guard in case the 'user' node isn't present
    try:
        user = appstate["NightlyBatch", "user"]
    except KeyError:
        user = "unknown"
    print("You cannot run this routine right now.")
    print(f"This routine is currently being run by user: {user}")
else:
    try:
        appstate["NightlyBatch"] = 1
        appstate["NightlyBatch", "user"] = getpass.getuser()
        # Option A: store a UNIX timestamp
        appstate["NightlyBatch", "starttime"] = int(time.time())
        # Option B (IRIS Horolog): appstate["NightlyBatch","starttime"] =
iris.cls("%SYSTEM.Util").Horolog()
        # --- main routine activity ---
        pass
    finally:
        appstate.kill(["NightlyBatch"])
        iris.unlock("", "^AppStateData", "NightlyBatch")
```

ObjectScript

```
  lock ^AppStateData("NightlyBatch"):0
  if '$TEST {
     write "You cannot run this routine right now."
     write !, "This routine is currently being run by user: "_^AppStateData("NightlyBatch","user")
     quit
  }
  set ^AppStateData("NightlyBatch")=1
  set ^AppStateData("NightlyBatch","user")=$USERNAME
  set ^AppStateData("NightlyBatch","starttime")=$h

  //main routine activity omitted from example

  kill ^AppStateData("NightlyBatch")
  lock -^AppStateData("NightlyBatch")
```

3.3 Example: Escalating Lock

The following example illustrates when escalating locks are created, how they behave, and how they are removed.

Suppose you have 1000 locks of the form `^MyGlobal(“sales”,”EU”,salesdate)` where *salesdate* represents individual dates. The lock table might look like this:

Owner	ModeCount	Reference	Directory
1284	Exclusive	^%SYS("CSP","Daemon")	c:\intersystems\iris\mgr\
26324	Exclusive	^ISC.LMFMON("License Monitor")	c:\intersystems\iris\mgr\
23400	Exclusive	^ISC.Monitor.System	c:\intersystems\iris\mgr\
23180	Exclusive	^TASKMGR	c:\intersystems\iris\mgr\
23948	Exclusive	^%cspSession("vgMJ4iLMCL")	c:\intersystems\iris\mgr\irislocaldata\
19776	Exclusive_e	^MyGlobal("sales","EU","2015-07-03")	c:\intersystems\iris\mgr\user\
19776	Exclusive_e	^MyGlobal("sales","EU","2015-07-04")	c:\intersystems\iris\mgr\user\
19776	Exclusive_e	^MyGlobal("sales","EU","2015-07-05")	c:\intersystems\iris\mgr\user\
19776	Exclusive_e	^MyGlobal("sales","EU","2015-07-06")	c:\intersystems\iris\mgr\user\
19776	Exclusive_e	^MyGlobal("sales","EU","2015-07-07")	c:\intersystems\iris\mgr\user\
19776	Exclusive_e	^MyGlobal("sales","EU","2015-07-08")	c:\intersystems\iris\mgr\user\
19776	Exclusive_e	^MyGlobal("sales","EU","2015-07-09")	c:\intersystems\iris\mgr\user\
19776	Exclusive_e	^MyGlobal("sales","EU","2015-07-10")	c:\intersystems\iris\mgr\user\
19776	Exclusive_e	^MyGlobal("sales","EU","2015-07-11")	c:\intersystems\iris\mgr\user\
19776	Exclusive_e	^MyGlobal("sales","EU","2015-07-12")	c:\intersystems\iris\mgr\user\

Notice the entries in the **Owner** column. This is the process that owns the lock. For owner 19776, the **ModeCount** column indicates that these are exclusive, escalating locks.

When the same process attempts to acquire an additional lock of the same form, InterSystems IRIS automatically escalates them. It removes the individual locks and replaces them with a single lock at the parent level: `^MyGlobal("sales","EU")`. Now the lock table might look like this:

Owner	ModeCount	Reference	Directory
1284	Exclusive	^%SYS("CSP","Daemon")	c:\intersystems\iris\mgr\
26324	Exclusive	^ISC.LMFMON("License Monitor")	c:\intersystems\iris\mgr\
23400	Exclusive	^ISC.Monitor.System	c:\intersystems\iris\mgr\
23180	Exclusive	^TASKMGR	c:\intersystems\iris\mgr\
23948	Exclusive	^%cspSession("vgMJ4iLMCL")	c:\intersystems\iris\mgr\irislocaldata\
19776	Exclusive/1001E	^MyGlobal("sales","EU")	c:\intersystems\iris\mgr\user\

The **ModeCount** column now shows a shared, escalating lock with a count of 1001.

Some key effects of this escalation:

- All child nodes of `^MyGlobal("sales","EU")` are now implicitly locked, following the basic rules for array locking.
- The lock table no longer contains information about which child nodes of `^MyGlobal("sales","EU")` were specifically locked, which affects how you remove locks.

If the same process continues to add more lock names of the form `^MyGlobal("sales","EU",salesdate)`, the lock table increments the lock count on `^MyGlobal("sales","EU")`. The lock table might then look like this:

Owner	ModeCount	Reference	Directory
1284	Exclusive	^%SYS("CSP","Daemon")	c:\intersystems\iris\mgr\
26324	Exclusive	^ISC.LMFMON("License Monitor")	c:\intersystems\iris\mgr\
23400	Exclusive	^ISC.Monitor.System	c:\intersystems\iris\mgr\
23180	Exclusive	^TASKMGR	c:\intersystems\iris\mgr\
23948	Exclusive	^%cspSession("vgMJ4iLMCL")	c:\intersystems\iris\mgr\irislocaldata\
19776	Exclusive/1026E	^MyGlobal("sales","EU")	c:\intersystems\iris\mgr\user\

The **ModeCount** column indicates that the lock count for this lock is now 1026.

To remove these locks, your application should continue releasing locks for specific child nodes. For example, suppose that your code removes the locks for `^MyGlobal("sales","EU",salesdate)` where *salesdate* corresponds to any date in 2011 — thus removing 365 locks. The lock table now looks like this:

Owner	ModeCount	Reference	Directory
1284	Exclusive	^%SYS("CSP","Daemon")	c:\intersystems\iris\mgr\
26324	Exclusive	^ISC.LMFMON("License Monitor")	c:\intersystems\iris\mgr\
23400	Exclusive	^ISC.Monitor.System	c:\intersystems\iris\mgr\
23180	Exclusive	^TASKMGR	c:\intersystems\iris\mgr\
23948	Exclusive	^%cspSession("vgMJ4iLMCL")	c:\intersystems\iris\mgr\irislocaldata\
19776	Exclusive/660E	^MyGlobal("sales","EU")	c:\intersystems\iris\mgr\user\

Even though the number of locks is now below the threshold (1000), the lock table does not list individual entries for the child-level locks. The parent node `^MyGlobal("sales","EU")` remains explicitly locked until 661 more child locks are removed.

Important: There is a subtle point to consider, related to the preceding discussion. It is possible for an application to "release" locks on array nodes that were never locked in the first place, thus resulting in an inaccurate lock count for the escalated lock and possibly releasing it before it is desirable to do so.

For example, suppose that the process locked nodes in `^MyGlobal("sales","EU",salesdate)` for the years 2010 through the present. This would create more than 1,000 locks, and the lock would be escalated, as planned. Suppose that a bug in the application removes locks for the nodes for the year 1970. InterSystems IRIS would permit this action even though those nodes were not previously locked, and it would decrement the lock count by 365. The resulting lock count would not be an accurately reflect the desired locks. If the application then removed locks for other years, the escalated lock could be unexpectedly removed early.

3.4 Example: Lock with Retry on Timeout

When you use a timeout with the **LOCK** in ObjectScript or `iris.lock()` in Python, the system will wait for the specified number of seconds before either acquiring the lock or giving up. You can check whether the lock was acquired and choose to retry if needed.

This is useful in cases when you expect temporary contention and want to retry the lock a few times before giving up entirely.

Python

```
retries = 3
for i in range(retries):
    try:
        iris.lock("", 2, "^MyResource")  # 2-second timeout
    except Exception as e:
        if i == retries - 1:
            raise
        # Lock not acquired - try again
        continue
    else:
        try:
            do_something()
        finally:
            iris.unlock("", "^MyResource")
        break
```

ObjectScript

```
SET retries = 3
FOR i=1:1:retries {
   LOCK +^MyResource:2
   IF $TEST {
       ; Lock acquired
       DO DoSomething()
       LOCK -^MyResource
       QUIT
   }
   ; Lock not acquired - try again
}
```

In either case, you are attempting to acquire the lock with a 2 second timeout. If the lock is not acquired, the code waits and retries up to the specified number of times.

This retry pattern can help reduce the chance of a process failing due to temporary lock contention.

3.5 Example: Timed Lock

This example demonstrates how to use a lock with a timeout. The process attempts to acquire a lock on `^a(1)` and waits up to five seconds. If the lock is acquired, the code modifies the global and releases the lock before committing the transaction. If the lock cannot be acquired within the timeout, the transaction is rolled back, and the process exits early.

Python

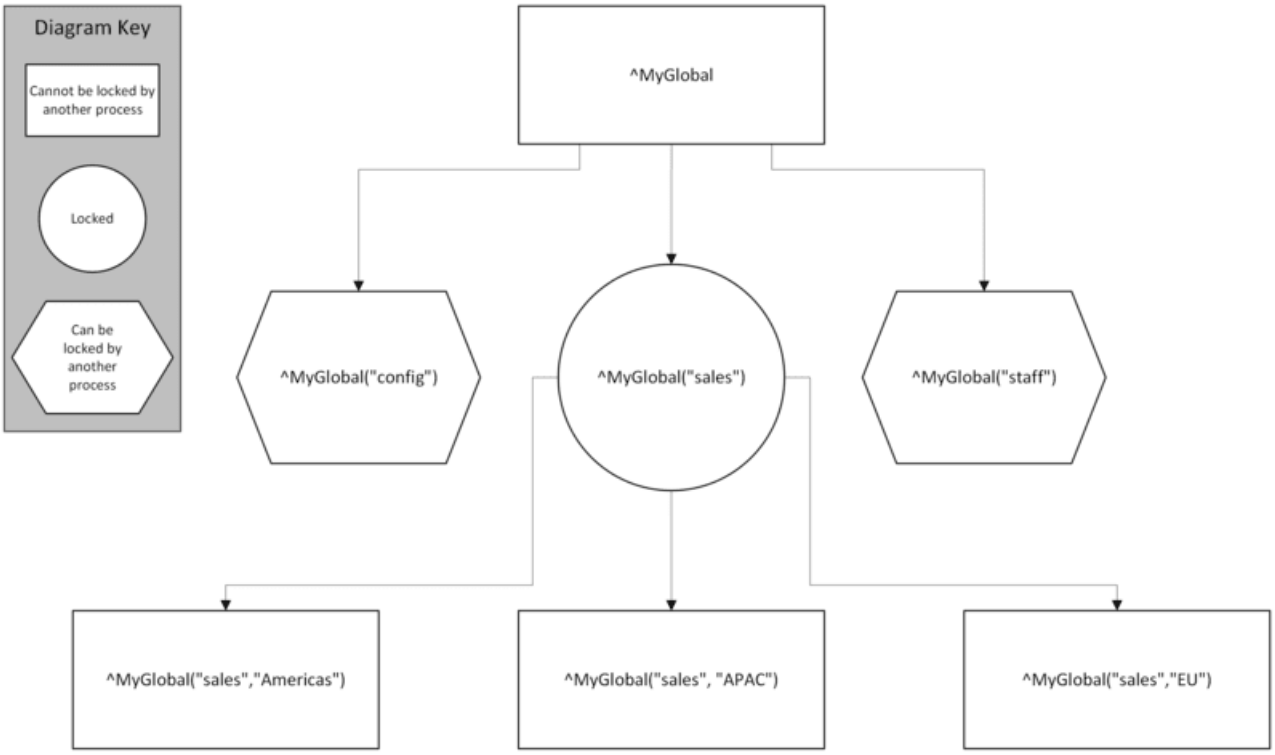
```
import iris
from iris import IRISTimeoutError

def timed_lock_example():
    try:
        print("Starting transaction")
        iris.tStart()

        # Attempt to acquire lock on ^a(1) with a timeout of 5 seconds
        iris.lock("", 5, "^a", 1)
        print("Lock acquired. Modifying ^a(1)")
        iris.gref("^a").set([1], 100)

        iris.unlock("", "^a", 1)
        print("Lock released.")

        iris.tCommit()
        print("Transaction committed.")
    except IRISTimeoutError:
        print("Could not acquire lock within timeout. Exiting early.")
        iris.tRollbackOne()
    except Exception as e:
        print(f"Error occurred: {e}")
        iris.tRollbackOne()
```

3.6 Example: Locking Arrays and Subnodes

When you lock an array, you can lock either the entire array or one or more nodes in the array. When you lock an array node, other processes are blocked from locking any node that is subordinate to that node. Other processes are also blocked from locking the direct ancestors of the locked node. Though they themselves are not locked, subordinate and direct ancestors of locked nodes are not accessible in this state and are considered implicitly locked. Implicit locks are not included in the lock table and thus do not affect its size.

The following figure shows an example:

Figure 3–1: Arrays in Locking



The InterSystems IRIS lock queuing algorithm queues all locks for the same lock name in the order received, even when there is no direct resource contention. For an example and details, see Queuing of Array Node Locks.

3.7 Example: Deferred Unlock

This example demonstrates a deferred unlock, which keeps a lock active until the current transaction is committed or rolled back. In this case, the lock on `^a(1)` is released using a deferred unlock, meaning it will remain in place until **TCOMMIT** is called. You can observe this behavior by viewing the lock table while the transaction is still in progress.

Note: Python does not support I or D lock-type codes.

ObjectScript

```
 TRY {
   TSTART
   LOCK +^a(1)               ; acquire as normal
   WRITE "Lock held. Scheduling deferred unlock.",!
   LOCK -^a(1)#"D"           ; schedule unlock at transaction end
   HANG 10                   ; verify it's still held in the Lock Table
   TCOMMIT                   ; deferred unlock happens here
   WRITE "Transaction committed; lock released.",!
} CATCH ex {
  WRITE "Error: ", ex.DisplayString(),!
  TROLLBACK
}
```

3.8 Example: Immediate Unlock

This example shows how to use an immediate unlock, which removes the lock as soon as the unlock call is issued, even if the transaction is still in progress. In this case, the lock on `^a(1)` is explicitly released before the transaction ends, as verified by checking the lock table before the commit occurs.

Note: Python does not support I or D lock-type codes.

ObjectScript

```
TRY {
 TSTART
 LOCK +^a(1)               ; acquire as normal
 WRITE "Lock acquired. Immediately releasing (inside a transaction).",!
 LOCK -^a(1)#"I"           ; immediate unlock (does not wait for TCOMMIT)
 HANG 10                   ; check Lock Table: ^a(1) is no longer held
 TCOMMIT
} CATCH ex {
 WRITE "Error: ", ex.DisplayString(),!
 TROLLBACK
}
```

4

Managing the Lock Table

InterSystems IRIS maintains a system-wide, in-memory table that records all current locks and the processes that own them. This table, the *lock table*, is accessible via the Management Portal, where you can view the locks and (in rare cases, if needed) remove them. This topic discusses tools for viewing and managing the lock table in InterSystems products. (Also see Monitoring Locks.)

4.1 About the Lock Table

As mentioned, InterSystems IRIS maintains an in-memory table that tracks all current locks and the processes that own them. Through the Management Portal, you can view and, when necessary, remove the locks.

The lock table has a configurable maximum size. If the Lock Table exceeds that size, you will receive a message in the messages log notifying you about the capacity being reached. Find out more about this in *Locking and Concurrency Control*.

4.2 Viewing Locks in the Lock Table

- Viewing Locks in the Management Portal
- Viewing Locks with ^LOCKTAB
- Viewing Locks Programmatically

4.2.1 Viewing Locks in the Management Portal

You can view all of the locks currently held or requested (waiting) system-wide using the Management Portal. From the Management Portal, select **System Operation**, select **Locks**, then select **View Locks**. The **View Locks** window displays a list of locks (and lock requests) in alphabetical order by directory (**Directory**) and within each directory in collation sequence by lock name (**Reference**). Each lock is identified by its process id (**Owner**), displays the user name that the operating system gave to the process when it was created (**OS User Name**), and has a **ModeCount** (lock mode and lock increment count). You may need to use the **Refresh** icon to view the most current list of locks and lock requests. For further details on this interface see Monitoring Locks.

4.2.1.1 Interpreting ModeCount Values in the Lock Table

ModeCount can indicate a held lock by a specific **Owner** process on a specific **Reference.** The following are examples of **ModeCount** values for held locks:

Note: Some ModeCount values reflect ObjectScript-specific locking behavior. Python locks are always incremental, so some values have no direct Python equivalent.

ModeCount	Description
Exclusive	An exclusive lock, non-escalating (`iris.lock("", 0, "^a", 1)` in Python or `LOCK +^a(1)` in ObjectScript).
Shared	A shared lock, non-escalating (`iris.lock("S", 0, "^a", 1)` in Python or `LOCK +^a(1)#"S"` in ObjectScript).
Exclusive_e	An exclusive escalating lock (`iris.lock("E", 0, "^a", 1)` in Python or `LOCK +^a(1)#"E"` in ObjectScript).
Exclusive_E	An exclusive/shared escalated lock, as a result of the number of locks on various nodes of this global exceeding the lock threshold.
Shared_e	A shared escalating lock (`iris.lock("SE", 0, "^a", 1)` in Python or `LOCK +^a(1)#"SE"` in ObjectScript).
Shared_E	A shared escalated lock, as a result of the number of locks on various nodes of this global exceeding the lock threshold.
Exclusive->Delock	An exclusive lock in a delock state. The lock has been unlocked, but release of the lock is deferred until the end of the current transaction. This can be caused by either a standard unlock (`iris.unlock(['^a(1)'])` in Python or `LOCK -^a(1)` in ObjectScript). In ObjectScript, it can also be caused by a deferred unlock (`LOCK -^a(1)#"D"`).
Exclusive,Shared	Both a shared lock and an exclusive lock (applied in any order). Can also specify escalating locks; for example, Exclusive_e,Shared_e.
Exclusive/*n*	An incremented exclusive lock (`iris.lock("", 0, "^a", 1)` in Python (all locks created with Python are incremental by default) or `LOCK +^a(1)` issued *n* times in ObjectScript). If the lock count is 1, no count is shown (but see below). Can also specify an incrementing shared lock; for example, Shared/2.
Exclusive/*n*->Delock	An incremented exclusive lock in a delock state. All of the increments of the lock have been unlocked, but release of the lock is deferred until the end of the current transaction. Within a transaction, unlocks of individual increments release those increments immediately; the lock does not go into a delock state until an unlock is issued when the lock count is 1. This ModeCount value, an incremented lock in a delock state, occurs when all prior locks are unlocked by a single operation, either by an argumentless **LOCK** command or a lock with no lock operation indicator (`LOCK ^xyz(1)` in ObjectScript).
Exclusive/1+1e	Two exclusive locks, one non-escalating, one escalating. Increment counts are kept separately on these two types of exclusive locks. Can also specify shared locks; for example, Shared/1+1e.
Exclusive/*n*,Shared/*m*	Both a shared lock and an exclusive lock, both with integer increments.

A held lock **ModeCount** can, of course, represent any combination of shared or exclusive, escalating or non-escalating locks — with or without increments. An Exclusive lock or a Shared lock (escalating or non-escalating) can be in a Delock state.

ModeCount can indicate a process waiting for a lock, such as `WaitExclusiveExact`. The following are **ModeCount** values for waiting lock requests:

ModeCount	Description
WaitSharedExact	Waiting for a shared lock on exactly the same lock, either held or previously-requested. For example, `LOCK +^a(1,2)#"S"` or `iris.lock("S", 5, "^a", 1, 2)` is waiting on `^a(1,2)`.
WaitExclusiveExact	Waiting for an exclusive lock on exactly the same lock, either held or previously-requested. For example, `LOCK +^a(1,2)` or `iris.lock("", 5, "^a", 1, 2)` is waiting on `^a(1,2)`.
WaitSharedParent	Waiting for a shared lock on the parent of a held or previously-requested lock. For example, `LOCK +^a(1)#"S"` or `iris.lock("S", 5, "^a", 1)` is waiting on `^a(1,2)`.
WaitExclusiveParent	Waiting for an exclusive lock on the parent of a held or previously-requested lock. For example, `LOCK +^a(1)` or `iris.lock("", 5, "^a", 1)` is waiting on `^a(1,2)`.
WaitSharedChild	Waiting for a shared lock on the child of a held or previously-requested lock. For example, `LOCK +^a(1,2)#"S"` or `iris.lock("S", 5, "^a", 1, 2)` is waiting on `^a(1)`.
WaitExclusiveChild	Waiting for an exclusive lock on the child of a held or previously-requested lock. For example, `LOCK +^a(1,2)` or `iris.lock("", 5, "^a", 1, 2)` is waiting on `^a(1)`.

ModeCount indicates the lock (or lock request) that is blocking this lock request. This is not necessarily the same as **Reference**, which specifies the currently held lock that is at the head of the lock queue on which this lock request is waiting. **Reference** does not necessarily indicate the requested lock that is immediately blocking this lock request.

ModeCount can indicate other lock status values for a specific **Owner** process on a specific **Reference**. The following are these other **ModeCount** status values:

ModeCount	Description
LockPending	An exclusive lock is pending. This status may occur while the server is in the process of granting the exclusive lock. You cannot delete a lock that is in a lock pending state.
SharePending	A shared lock is pending. This status may occur while the server is in the process of granting the shared lock. You cannot delete a lock that is in a lock pending state.
DelockPending	An unlock is pending. This status may occur while the server is in the process of unlocking a held lock. You cannot delete a lock that is in a lock pending state.
Lost	A lock was lost due to network reset.

Select **Display Owner's Routine Information** to enable the **Routine** column, which provides the name of the routine that the owner process is executing, prepended with the current line number being executed within that routine.

Select **Show SQL Options**, and then select a namespace from the **Show SQL Table Names for Namespace** list, to enable the **SQL Table Name** column. This column provides the name of the SQL table associated with each process in the selected namespace. If the process is not associated with an SQL table, this column value is empty.

The **View Locks** window cannot be used to remove locks.

4.2.2 Viewing Locks with ^LOCKTAB

You can view current lock table entries using the **^LOCKTAB** utility in the %SYS namespace.

To display locks in read-only mode, use:

```
DO View^LOCKTAB
```

This form displays the current contents of the lock table; it does not provide options to modify or delete locks.

You can also use the full utility:

```
DO ^LOCKTAB
```

This displays the same lock information but also includes administrative commands. For details on deleting locks using this utility, see *Removing Locks*.

The display includes information about each lock, such as the owning process, lock mode, and any waiting processes. For example:

```
%SYS>DO ^LOCKTAB

                              Node Name: MYCOMPUTER
                  LOCK table entries at 07:22AM  01/13/2018
                16767056 bytes usable, 16774512 bytes available.

Entry Process     X#   S# Flg   W# Item Locked
   1) 4900         1              ^["^^c:\intersystems\iris\mgr\"]%SYS("CSP","Daemon")
   2) 4856         1              ^["^^c:\intersystems\iris\mgr\"]ISC.LMFMON("License Monitor")
   3) 5016         1              ^["^^c:\intersystems\iris\mgr\"]ISC.Monitor.System
   4) 5024         1              ^["^^c:\intersystems\iris\mgr\"]TASKMGR
   5) 6796         1              ^["^^c:\intersystems\iris\mgr\user\"]a(1)
   6) 6796        1e              ^["^^c:\intersystems\iris\mgr\user\"]a(1,1)
   7) 6796              2        1 ^["^^c:\intersystems\iris\mgr\user\"]b(1)Waiters: 3120(XC)
   8) 3120         2              ^["^^c:\intersystems\iris\mgr\user\"]c(1)
   9) 2024         1    1         ^["^^c:\intersystems\iris\mgr\user\"]d(1)

Command=>
```

In this display:

- The *X#* column lists exclusive locks held, with the number indicating the lock increment count. The “e” suffix indicates that the lock is defined as escalating. The “D” suffix indicates that the lock is in a delock state.
- The *S#* column lists shared locks held, with the number indicating the lock increment count. The “e” suffix indicates that the lock is defined as escalating. The “D” suffix indicates that the lock is in a delock state.
- The *Flg* column indicates whether the lock is in a pending state.
- The *W#* column shows the number of processes waiting for the lock.

As shown in the above display, process 6796 holds an incremented shared lock ^b(1). Process 3120 has one lock request waiting for this lock. The lock request is for an exclusive (X) lock on a child (C) of ^b(1).

Enter a question mark (`?`) at the `Command=>` prompt to display help for interpreting the output.

Enter `Q` to exit the utility.

4.2.3 Viewing Locks Programmatically

You can query lock table information programmatically using the %SYS.LockQuery class, which lets you read lock table information. The SYS.Lock class, available in the %SYS namespace, provides related administrative and configuration methods. These queries are useful for identifying lock owners, waiting processes, and potential lock conflicts when diagnosing concurrency issues.

For more information on these classes, see the class reference.

4.3 Deleting Locks

There are multiple ways to remove (delete) locks from the lock table as an administrative action.

Important: Rather than removing a lock, the best practice is to identify and then terminate the process that created the lock. Removing a lock can have a severe impact on the system, depending on the purpose of the lock.

4.3.1 Removing Locks in the System Management Portal

To remove (delete) locks currently held on the system, go to the Management Portal, select **System Operation**, select **Locks**, then select **Manage Locks**. For the desired process (**Owner**) click either **Remove** or **Remove All Locks for Process**.

Removing a lock deletes it from the lock table, regardless of lock type or increment level, and makes it available to other processes. For details on how this affects lock state and waiting processes, see *Locking and Concurrency Control*.

You can also remove locks using the **SYS.Lock.DeleteOneLock()** and **SYS.Lock.DeleteAllLocks()** methods.

Removing a lock requires WRITE permission. Lock removal is logged in the audit database (if enabled); it is not logged in messages.log.

For details on how lock removal affects lock state and queues, see *Locking and Concurrency Control*.

4.3.2 Removing Locks with ^LOCKTAB

You can remove (delete) locks using the **^LOCKTAB** utility in the %SYS namespace.

To access lock removal commands, run:

```
DO ^LOCKTAB
```

The **^LOCKTAB** utility provides interactive commands for:

- Deleting an individual lock
- Deleting all locks owned by a specified process
- Deleting all locks on the system

Enter a question mark (?) at the `Command=>` prompt to display the available commands and their usage.

Note: You cannot delete a lock that is in a lock pending state, as indicated by the *Flg* column (described in Viewing Locks with the ^LOCKTAB Utility).

For details on how lock removal affects lock behavior and waiting processes, see *Locking and Concurrency Control*.

Enter `Q` to exit the utility.

4.4 See Also

- Locking and Concurrency Control
- Details of Lock Requests and Deadlocks